Vibe Coding Safely — 15-Minute Talk Script

Oct 18, 2026 · @Adam

Run sheet

Five parts in 15 minutes, each one following Incident, then Why, then Prevent. There are four on-screen demos, and each one needs a screenshot backup in case the Wi-Fi or a laptop fails.

#	Part	Clock	Incident	On screen
1	A working app is not a safe app	0:00–2:00	EnrichLead shutdown	Talk only
2	Secrets leak fast	2:00–5:00	GitGuardian leak data	DevTools key hunt, then the .env fix
3	Your database is open by default	5:00–9:00	Lovable CVE-2025-48757	Supabase RLS off, then on
4	Don't blindly trust what the AI installs or runs	9:00–13:00	huggingface-cli, Equifax, Replit	npm audit
5	Privacy and the checklist	13:00–15:00	Tea app	Attacker prompt

How to deliver it. The lines in quotes are written to be said out loud. Learn the idea and say it your own way, because reading word-for-word kills the room. Lines marked **[ON SCREEN]** are what you show.

Demo setup (do this the day before):

- [ ] A toy web app with a **fake or already-revoked** API key hardcoded in the frontend. Never use a real key on stage.
- [ ] A Supabase project with a `students` table filled with fake names and emails, with RLS **off**.
- [ ] A folder with a `package.json` pinning an old version, for example `"axios": "0.21.1"`. Run `npm audit` once beforehand to confirm the warnings appear.
- [ ] A screenshot of every demo step, in case anything breaks live.

Part 1: A working app is not a safe app (0:00–2:00)

Incident: EnrichLead, March 2025

> "Quick show of hands. Who here has asked an AI to build something, and it just worked on the first try? Keep your hand up if you shipped it."
>
> "In March 2025 a founder named Leo Acevedo did exactly that. He built a sales-lead SaaS called EnrichLead using Cursor, and posted proudly that it had zero hand-written code. People paid for it."
>
> "Days later he posted again: he was under attack. People bypassed his subscriptions and used the app for free. His API keys were maxed out on his bill. Junk was being written into his database. He shut the whole thing down."

Why it happened

> "The AI didn't do anything evil. It did exactly what he asked: make it work. Nobody asked it to make it safe."
>
> "And because he never read the code, he couldn't know what it exposed. Then he announced it publicly, which made him a target."

How to prevent it (talk only)

> "So here's the one question I want you to leave with. Stop asking only 'does it work?' Also ask, 'who *else* can make it work?' Who else can read this data, call this API, or skip this paywall?"
>
> "In the next 13 minutes I'll show you the four places that question usually fails, and how to fix each one in a few minutes."

Transition: *"Let's start with the easiest mistake, which is also exactly what happened to Leo."*

Part 2: Secrets leak fast (2:00–5:00)

Incident: millions of keys on public GitHub

> "Leo's API keys were sitting in his frontend code. He's not unusual."
>
> "GitGuardian scans public GitHub every year. In 2024 they found 23.8 million leaked secrets: passwords, API keys, database logins. In 2025 it jumped to about 29 million, and commits written with AI help leaked secrets at roughly double the normal rate."
>
> "The scariest number: 70% of the secrets leaked in 2022 were still working years later. People leak a key and never change it."

Why it happened

> "There are two ways keys escape. First, you put the key in frontend code. Anything sent to the browser is public, because anyone can open DevTools."
>
> "Second, you commit it to Git. Deleting the file later doesn't help, because it's still in the history. Bots scan new public commits constantly."
>
> "And the AI puts the key wherever it makes the code work fastest. That's usually the worst possible place."

How to prevent it [ON SCREEN]

1. **Find the leak.** Open the toy app, press F12, go to Sources, and search for `sk-` or `key`. The key shows up.

   > "This is all an attacker needs. No hacking skills, just F12."

2. **Move the key out of the frontend.** Put it in a `.env` file and call the API from a server route instead of the browser:

```
# .env  (never commit this file)
OPENAI_API_KEY=sk-...
```

```
# .gitignore
.env
```

```
// app/api/chat/route.js  (runs on the server, not in the browser)
const key = process.env.OPENAI_API_KEY;
```

3. **Point out the trap.** Variables starting with `NEXT_PUBLIC_` or `VITE_` are shipped *to the browser*.

   > "A .env file alone isn't safe. The key also has to be used only on the server."

4. **If it already leaked,** rotate the key in the provider's dashboard right away.

> "Deleting the commit is not a fix. Revoking the key is."

Transition: *"Keys are one kind of secret. But for most of your apps, the real treasure is the database."*

Part 3: Your database is open by default (5:00–9:00)

Incident: Lovable, CVE-2025-48757

> "Lovable is one of the most popular vibe-coding platforms. In 2025, a researcher named Matt Palmer scanned 1,645 apps from Lovable's own showcase."
>
> "About 1 in 10, 170 apps, had databases that anyone on the internet could read, and sometimes even write to. Inside were emails, home addresses, personal debt amounts, and API keys. It became an official security vulnerability, CVE-2025-48757."
>
> "It kept happening. In early 2026, one featured Lovable app exposed nearly 18,700 user records, including students from UC Berkeley and UC Davis. These are students like you, using an app built by someone like you."

Why it happened

> "Supabase and Firebase let the browser talk straight to the database using a public key. That key is *supposed* to be public. It's like the address of the building."
>
> "The lock on each table is called Row Level Security, or RLS. When AI tools create tables with raw SQL, RLS is often left off. No lock means anyone with the address walks in."
>
> "And hiding a button in your UI is not a lock. The attacker never uses your UI. They talk to your database directly."

How to prevent it [ON SCREEN]

1. **Show the open door.** In the browser console of the toy app, using only the public anon key:

```
const { data } = await supabase.from('students').select('*');
console.log(data);   // every student, every row
```

2. **Turn the lock on.** In the Supabase SQL editor:

```
ALTER TABLE public.students ENABLE ROW LEVEL SECURITY;
```

3. **Re-run step 1.** It now returns an empty list.

> "Same code, same key, nothing comes back."

4. **Add a real rule** so users see only their own rows:

```
CREATE POLICY "own rows only" ON public.students
  FOR SELECT USING (auth.uid() = user_id);
```

5. **Warn about the fake fix.** A policy with `USING (true)` means everyone can read everything again.

> "Before launch, open Supabase's Security Advisor, or your platform's security scan, and fix every red item."

Firebase users: the same idea applies to your Security Rules. Never ship `allow read, write: if true;`.

Transition: *"So far we've talked about code the AI wrote. Now let's talk about code the AI pulls in from other people."*

Part 4: Don't blindly trust what the AI installs or runs (9:00–13:00)

Three short stories in four minutes. Keep each one tight.

4A. Packages that don't exist (9:00–10:00)

Incident: huggingface-cli

> "AI models kept telling people to run `pip install huggingface-cli`. That package didn't exist. The real command is different."
>
> "A security researcher, Bar Lanyado from Lasso Security, registered that fake name with an empty package, as a harmless test. In three months it got over 30,000 downloads. Alibaba even copied the wrong command into one of its project READMEs."

Why it happened

> "AI invents package names that sound right. A 2025 study across 16 models found about 1 in 5 recommended packages didn't exist. And 43% of those fake names came back every single time, so attackers can predict them, register them first, and put malware inside. This attack is called slopsquatting."

How to prevent it (talk)

> "Before you install anything the AI suggests, spend 10 seconds on the npm or PyPI page. Check how old it is, how many weekly downloads it has, and whether it links to a real GitHub repo. If it was created last week and has 12 downloads, don't install it."

4B. Packages that are too old (10:00–11:30)

Incident: Equifax, 2017

> "Equifax is a credit agency. In March 2017, a serious flaw in a library they used, Apache Struts, was announced, and a fix was released the same day. Equifax didn't apply it."
>
> "Two months later attackers walked in through that exact hole and stole data on 147 million people. It was one known bug in one old library, with the fix already available."

Why it happened

> "Two reasons apply to you. First, AI often writes version numbers from memory, from whenever it was trained, so it may pin a version that's known to be vulnerable. Second, even a perfect app goes stale. A library that's safe today gets a new CVE next month, and nobody goes back to check."

How to prevent it [ON SCREEN]

1. Show `package.json` with an old pinned version, for example `"axios": "0.21.1"`.
2. Run:

```
npm audit
```

3. Let the red warnings sit on screen for a few seconds.

   > "Each line here is a published vulnerability. It took one second to find."

4. Run `npm audit fix` and show the clean result.

5. Mention the alternatives: Python users run `pip-audit`. On GitHub, turn on **Dependabot** in the repo's security settings so updates come to you automatically.

4C. Agents with too much power (11:30–13:00)

Incident: Replit and SaaStr, July 2025

> "Jason Lemkin, founder of SaaStr, was nine days into building an app with Replit's AI agent. He'd told it clearly: code freeze, don't change anything without asking."
>
> "The agent ran a database command anyway and wiped records for about 1,200 executives and 1,200 companies. Then it told him a rollback was impossible. That was wrong, and he recovered the data himself."

Why it happened

> "The agent was holding the keys to the real database. An instruction in a chat is not a permission. If the agent *can* delete production, one day it might."

How to prevent it (talk)

- Keep separate databases for development and production. Replit itself added this after the incident.
- Never give an agent production credentials.
- Turn on approval prompts for commands that delete, push or migrate.
- Make sure backups exist *before* you need them.

Transition: *"Everything so far is about attackers. The last part is about the people whose data you hold."*

Part 5: Privacy and the checklist (13:00–15:00)

Incident: Tea app, July 2025

> "Tea was a dating-safety app for women. To sign up, users sent a selfie, and early on, a photo of their ID. The app said it would delete those photos after verification."
>
> "It didn't. About 72,000 images, including roughly 13,000 selfies and ID photos, sat in a cloud storage bucket with no lock. Someone found it, and the photos ended up on 4chan. A second leak a few days later exposed over a million private messages."
>
> "One honest note: Tea's code was written *before* vibe coding took off. That's the point. This isn't an AI problem. Data you keep is data you can leak."

Why it happened

> "Three mistakes stacked up. They collected very sensitive data. They kept it after they no longer needed it. Then they stored it without access rules. Remove any one of those and the story is much smaller."

How to prevent it (talk, then [ON SCREEN])

> "For Malaysians: if your app collects names, emails, IC numbers or matric numbers, the Personal Data Protection Act applies to you, even as a student. Since 2025, breaking its principles can mean a fine of up to RM1 million, and serious breaches must be reported to the Commissioner."
>
> "So for every field on your sign-up form, ask: do I actually need this? If not, don't collect it. If you only need it briefly, delete it after."

[ON SCREEN]: paste the toy app's code into any AI and send:

```
Act as an attacker. List every way someone could steal data,
bypass payment, or run up my bills in this code.
Rank them by severity and show the fix for each.
```

> "Watch. It finds the hardcoded key and the open table, the same things we just fixed by hand. The AI that wrote the bug can find the bug, but only if you ask."

Closing slide: 5 checks before you ship

- [ ] No secrets in frontend code or Git history. Keys stay on the server.
- [ ] RLS or security rules on every table and storage bucket.
- [ ] Every package is real and maintained, and `npm audit` / `pip-audit` comes back clean.
- [ ] AI agents never hold production credentials, and backups exist.
- [ ] Collect only the data you need, and delete it when you're done.

> "Vibe code as much as you want. It's a great way to learn. Just don't ship until you can answer one question: who *else* can use this? Thank you."

Sources

Every number in the script comes from these pages. Put the short link on the slide where you use it.

Part	Claim	Source
1	EnrichLead built with Cursor, keys exposed, shut down March 2025	Vibe Graveyard, Indie Hackers
2	23.8M secrets leaked in 2024; 70% of 2022 leaks still active	GitGuardian
2	~29M in 2025; AI-assisted commits leak ~2x the baseline	TechRadar
3	170 of 1,645 Lovable apps exposed, CVE-2025-48757	Superblocks, RapidDev guide
3	18,697 records incl. UC Berkeley / UC Davis users, Feb 2026	Bastion
4A	huggingface-cli: 30,000+ downloads; ~20% hallucination rate, 43% repeat	Wikipedia: Slopsquatting, concept summary of USENIX 2025 study
4B	Equifax: Struts CVE-2017-5638 patch released March 2017, not applied, 147M people	Digital Guardian, Bad Technology
4C	Replit agent wiped 1,206 execs / ~1,196 companies during code freeze; data recovered	The New Stack, heise
5	Tea: ~72,000 images, ~13,000 selfies/IDs; code pre-dates vibe coding	Simon Willison, The Record
5	PDPA fines up to RM1M from 1 April 2025; breach notification	Legal 500 / Tay & Partners, Rajah & Tann